

VIT Bhopal University

A Project Report Submitted for
CSE2006 - Programming in Java

SLOT- B14+D21

BANK ACCOUNT & TRANSACTION MANAGEMENT SYSTEM

A Java-based Command-Line Banking & Transaction Simulation System

Submitted By

SANDEEP GANESH

Registration No: 25BAI11622

B.Tech CSE (Artificial Intelligence and Machine Learning)

Under the Guidance of

Dr. S. Devaraju

Academic Year: 2026 – 2027

Date of Submission: 14 September 2026

1. Introduction

All banking applications have common problems: how to represent an account and its owner, how to move an amount of money from one account to another without losing or duplicating it, how to process multiple transactions in parallel without corrupting shared data and how to keep an immutable, auditable log of what happened. The Bank Account & Transaction Management System can be implemented as a simple self-contained Java application to constitute the whole banking operation. The input logic, input validation, business rules, concurrent processing and permanent storage are implemented in a single runnable program.

To this end, it models two real-world kinds of bank account (a Savings account with a minimum balance and a Current account, which allows an overdraft up to a specified limit) and handles all deposits, withdrawals and transfers asynchronously from a pool of threads. Transfers use per-account locks and a strict locking order to avoid lost updates and deadlocks. Three custom checked exceptions indicate invalid accounts, insufficient funds, and negative amounts, respectively. The exceptions ensure that all failures leave the account data in a valid state, and that all operations (successful or not) are recorded in a transaction log.

The project also presents abstraction, inheritance, interfaces and its own custom exceptions. It uses the `java.util.concurrent` package correctly, its well-layered architecture makes it extremely easy to swap the storage layer for a real database server, and the built-in simulation of concurrent load provides immediate, repeatable evidence that whatever final implementation is chosen does preserve account balances across many concurrent transfers, with no special setup needed other than the JDK. Inheritance is also a highly practical and useful feature of this program. Account implementations can introduce nice features like casting warnings about concurrency problems (like lost updates and deadlocks) into repeatable experiments just by removing the lock-ordering logic and re-running the load test. The audit-trail discipline enforces the kind of behavior from financial and stateful systems that are expected to maintain that kind of detail for accountability purposes. And finally, the project's clean separation of concerns makes it easy to add further systems. For example it would be trivial to build a REST API, a relational database, or to introduce multiple types of accounts, without having to rewrite the core transactional work of transferring money between accounts.

It is easy to deploy and use, since it uses only the Java standard library and not a Java framework or network connection. It does not become corrupted by concurrency problems, because it uses per-account locks and ordered lock acquisition. The cost of development is nearly zero, the console menu requires no training, and state (game progress) is maintained across restarts without manual configuration.

The Bank Account & Transaction Management System is written using Java Standard Edition (JDK 17 or higher) and depends on the following packages: `java.io/java.nio` for persistent storage, `java.math.BigDecimal` for monetary values, `java.time` for timestamps, `java.util.concurrent` (`ExecutorService`, `Future`, `Callable`, `ReentrantLock`) for asynchronous, thread-safe execution, and `java.util` (`UUID`, `Scanner`, `ConcurrentHashMap`) for unique identifiers, console input, and in-memory data caches. It does not rely on any framework, database, or server, and should be able to compile on a basic JDK distribution and run using just the plain `javac/java` compiler/runtime. It is organized into the following architectural layers: a console command-line layer, service layer (with a thread-safe account cache), engine layer (which handles asynchronous deposits/withdrawals/transfers), persistence layer (with flat text file persistence) and model layer (with accounts and transactions). An abstract account class implementing a `Transactable` interface illustrates abstraction, inheritance and polymorphism: different account types can use subclasses (`Savings` and `Current`) to customize withdrawal business logic in the engine layer. All business operations are processed on the same thread pool and return a `Future`. Locks are acquired on a per-account basis in a defined order (ensuring that deadlocks cannot occur). Three checked exceptions allow business failures to be recorded rather than corrupting data or failing the application.

2. Problem Statement

Banks need to handle many customer transactions at the same time, such as deposits, withdrawals, and fund transfers, while making sure that account balances always remain correct. When two transactions access the same account at the same time, problems can occur if the operations are not handled properly. For example, one transaction may read an old balance while another transaction is updating it, which can result in an incorrect balance or a transfer where money is deducted from one account but not added to the other. This is a common race condition that can occur when multiple threads access shared data simultaneously.

The aim of this project is to develop a terminal-based Bank Account & Transaction Management System that demonstrates how such problems can be handled safely. The system supports different types of bank accounts and uses multithreading with a thread pool to process multiple transactions concurrently. It ensures that the total balance of all accounts remains unchanged during transfers, even when several transactions are running at the same time.

The system also handles invalid transactions properly. Requests such as transferring money from a non-existent account, withdrawing more than the allowed balance, exceeding an overdraft limit, or entering a negative amount are rejected with suitable error messages instead of affecting the account data or causing the program to crash.

Overall, the project demonstrates how core Java, OOP, exception handling, multithreading, Collections, and file-based I/O can be used together to build a simple and reliable banking system.

3. Functional Requirements

The system is divided into three main functional modules. Each module represents a specific set of activities available through the command-line menu of the BankApp class.

Module 1 — Account Management

- FR-1: The system allows users to create a new SAVINGS or CURRENT account. A savings account can have a minimum balance and interest rate, while a current account can have an overdraft limit. This can be done using menu option 1 (Create Account).
- FR-2: Whenever a new account is created, its details are immediately saved in the accounts.txt file. This allows the account information to remain available even after restarting the application.
- FR-3: Users can view all the accounts available in the system through menu option 6 (View All Accounts). The displayed information includes the account number, account holder's name, account type, and current balance.
- FR-4: The system applies different withdrawal rules based on the account type. A SAVINGS account cannot be withdrawn below its minimum balance, while a CURRENT account can go below zero only within its allowed overdraft limit.

Module 2 — Concurrent Transaction Processing

- FR-5: Users can make deposit and withdrawal requests by providing the account number and the required amount through menu options 2 and 3.
- FR-6: The system also supports fund transfers between two accounts using menu option 4. The amount is deducted from the source account and added to the destination account as one logical transaction.
- FR-7: Deposits, withdrawals, and transfers are processed using a fixed thread pool through ExecutorService. Each operation runs as a separate task and returns a Future<Transaction> containing the transaction result.

- FR-8: The system prevents incorrect balances when multiple transactions access the same account at the same time. This is tested using menu option 7 (Simulate Concurrent Load Test), where multiple transfers are performed simultaneously and the total balance before and after the test is compared.
- FR-9: Every transaction attempt is recorded, whether it is successful or failed. The record contains a unique transaction ID, timestamp, transaction status (SUCCESS/FAILED), and a short explanation of the result.

Module 3 — Statement, Reporting & Persistence

- FR-10: Users can view the complete transaction history of a particular account using menu option 5 (View Account Statement). The statement includes transactions where the account was either the sender or receiver.
- FR-11: The system provides an option to export an account statement as a CSV file using the account number as part of the filename, such as <accountNumber>_statement.csv.
- FR-12: Account and transaction information is stored in simple pipe-delimited text files, namely accounts.txt and transactions.txt. The system uses Java's java.io and java.nio features for file handling, so it can run directly from a terminal without requiring external software or libraries.

Operation	Input	Output
Create Account	Account number, holder name, opening balance, type (SAVINGS/CURRENT)	Confirmation line with the account's to String() summary
Deposit / Withdraw	Account number, amount	Transaction result (SUCCESS/FAILED + remarks) via Future.get()
Transfer	From account, to account, amount	Transaction result; both balances updated atomically
View Statement	Account number	List of Transaction rows; optional CSV export
Concurrent Load Test	Number of simulated transfers	Total balance before/after and a conservation check (true/false)

4. Non-functional Requirements

1. Concurrency & Performance

- Parallel Execution: Achieved via a fixed thread pool (Executors.newFixedThreadPool(8)) within the TransactionEngine, allowing deposits, withdrawals, and transfers to execute concurrently.
- Granular Locking: Concurrency control is isolated at the individual Account level rather than utilizing a global lock, maximizing throughput.
- Deadlock Prevention: The TransferTask enforces a strict, deterministic locking order based on account numbers to guarantee system stability during concurrent transfers.

2. Data Integrity & Security

- Financial Reliability: Thread safety is maintained using java.util.concurrent.locks.ReentrantLock per account, ensuring the total system balance remains immutable across concurrent operations.

- **Input Validation:** The InputHelper layer acts as a strict gatekeeper, sanitizing user inputs and rejecting zero or negative values before they interact with downstream business logic.

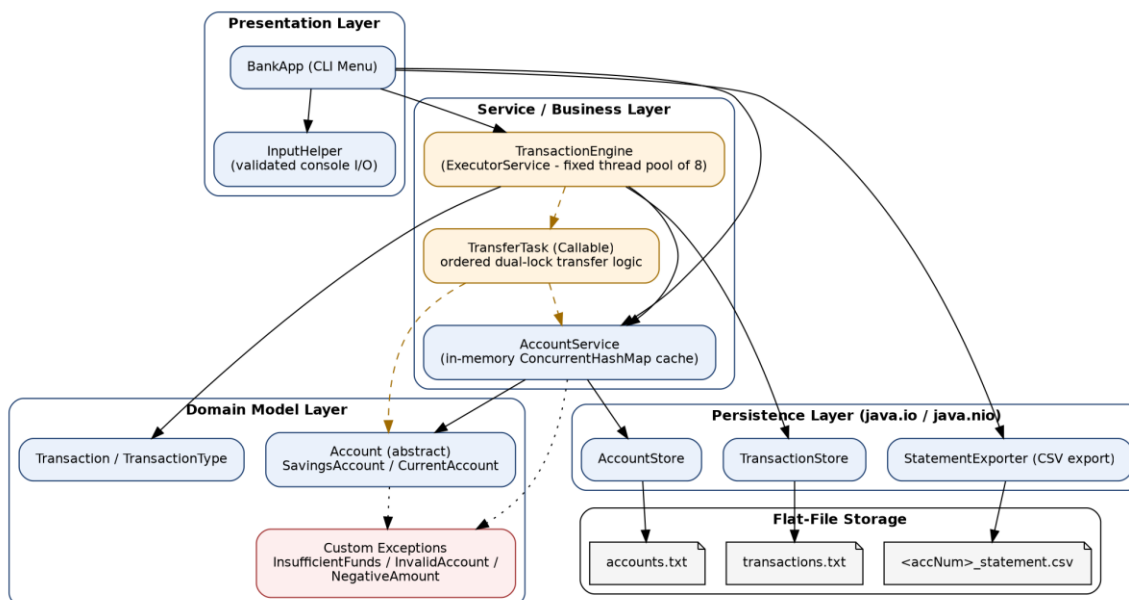
3. System Maintainability & Portability

- **Layered Architecture:** Responsibilities are strictly segregated into Presentation, Service, Domain, and Persistence layers. This decoupling ensures that modifications to storage or UI components do not impact unrelated subsystems.
- **Zero-Dependency Deployment:** The system relies exclusively on the standard java.io and java.nio libraries. It requires no external database engines, drivers, or build tools, compiling and executing via standard terminal commands.

4. Fault Tolerance & Auditability

- **Exception Management:** Foreseeable failures (e.g., insufficient funds, invalid accounts) are intercepted at the task boundary using targeted, checked exceptions. This prevents application crashes and gracefully records the failure state.
- **Comprehensive Auditing:** Every transaction attempt is appended as a pipe-delimited record to transactions.txt. Failed attempts are preserved alongside successful ones, capturing the exact error context for troubleshooting.
- **Resource Lifecycle:** The application prevents resource leaks by utilizing try-with-resources blocks for all file I/O operations and executing a clean shutdown sequence for the executor service upon exit.

5. System Architecture



The 4 Layers of the System

1. Presentation Layer (The Face)

- BankApp: Runs a simple, numbered command-line menu loop that users see and interact with.
- Input Helper: Packages up the Java Scanner tool. It automatically checks user inputs and asks again if they type a typo, making sure the app gets valid words, numbers, and dollar amounts.

2. Service / Business Layer (The Brains)

- Account Service: Holds the master list of accounts safely in live memory using a Concurrent HashMap.
- Transaction Engine: Manages a background worker pool (ExecutorService). It handles deposits, withdrawals, and transfers as background tasks so the app doesn't freeze up.
- Transfer Task: A worker task that safely locks two accounts at once, moves the money, and returns a record of the transaction.

3. Domain Model Layer (The Rules)

- Account: An abstract blueprint that dictates how accounts behave.
- Savings Account & Current Account: The specific types of bank accounts.
- Transaction & Transaction Type: The data record and labels used to track money moves.
- Custom Exceptions: Three specific error checkers built to handle bank-specific problems.

4. Persistence Layer (The Storage)

- Account Store & Transaction Store: Save and read data from plain text files. They use a strict lock system so two actions cannot mess up the files at the exact same time.
- Statement Exporter: Turns a list of transactions into a clean CSV file.

How Data Moves

When a user types a command, data moves straight down: Menu → Service → Account

Rules → File Storage.

When the bank processes a transaction, it sends a Future object back up to the menu. Because it processes in the background, the user can keep typing and using the menu without waiting for the file to finish saving.

6. Design Diagrams

Use Case Diagram

Two actors interact with the system: the Customer, who performs everyday banking operations, and the Bank Operator, who additionally audits accounts and validates system behaviour under concurrent load. Deposit, Withdraw, and Transfer all include a common Validate Transaction step (amount and account existence checks); viewing a statement can optionally extend into exporting it as CSV.

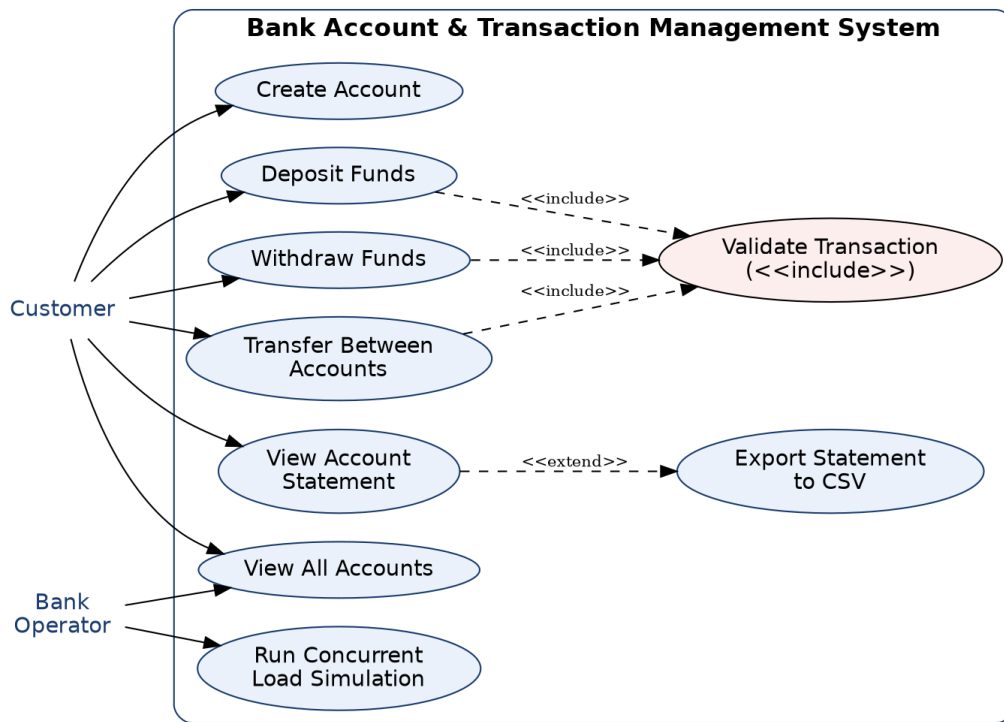


Figure 6.1 — Use case diagram

Workflow / Activity Diagram

The diagram below traces one full menu cycle for a Transfer request, from CLI input through validation, locking, balance mutation, persistence, and logging, back to the menu — including both failure branches (invalid account, and negative amount / insufficient funds). Because the flow is long and strictly sequential, it is presented across two figures (6.2a continues into 6.2b).

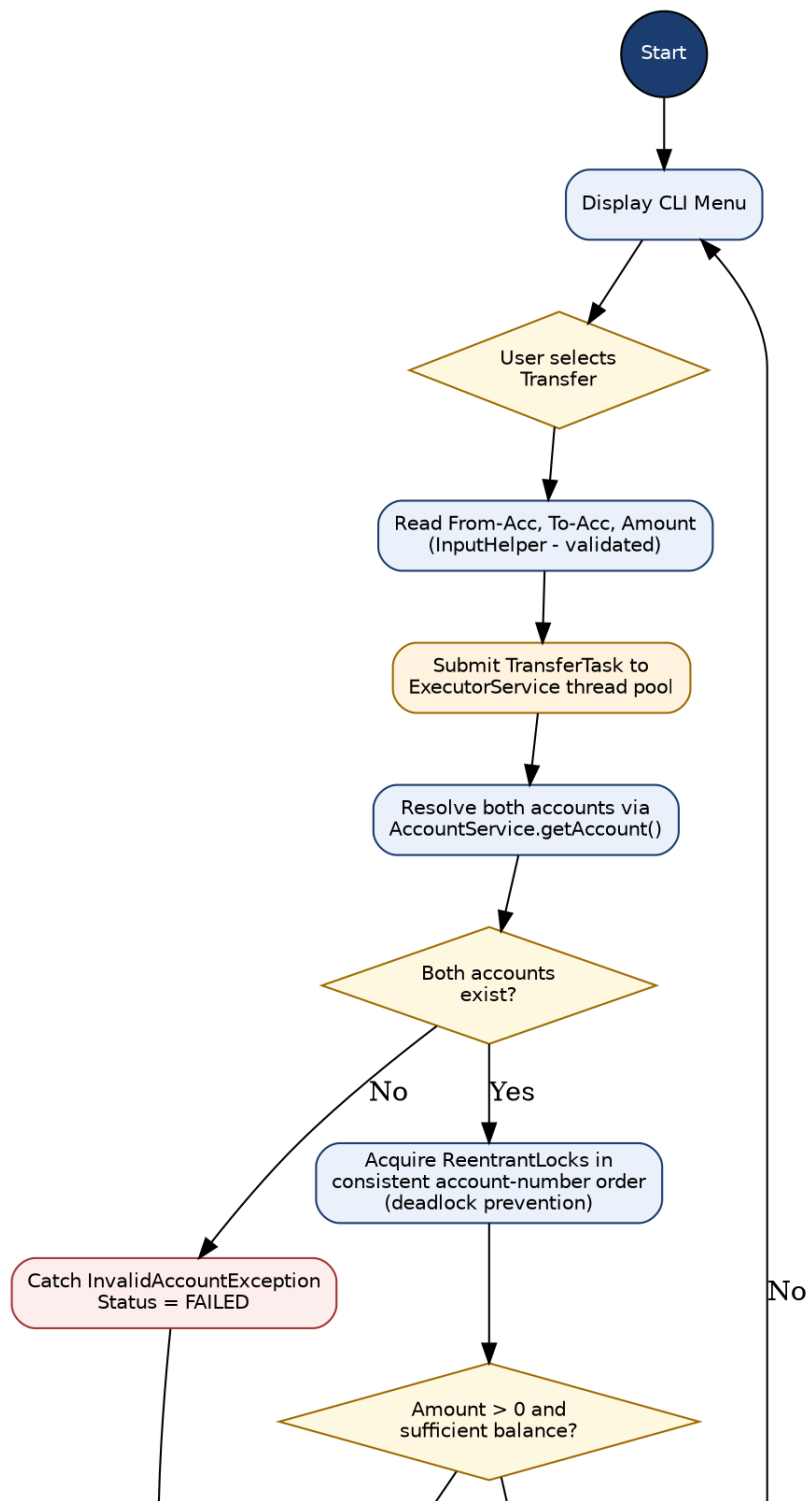


Figure 6.2a — Workflow diagram for a Transfer request

Workflow / Activity Diagram (continued)

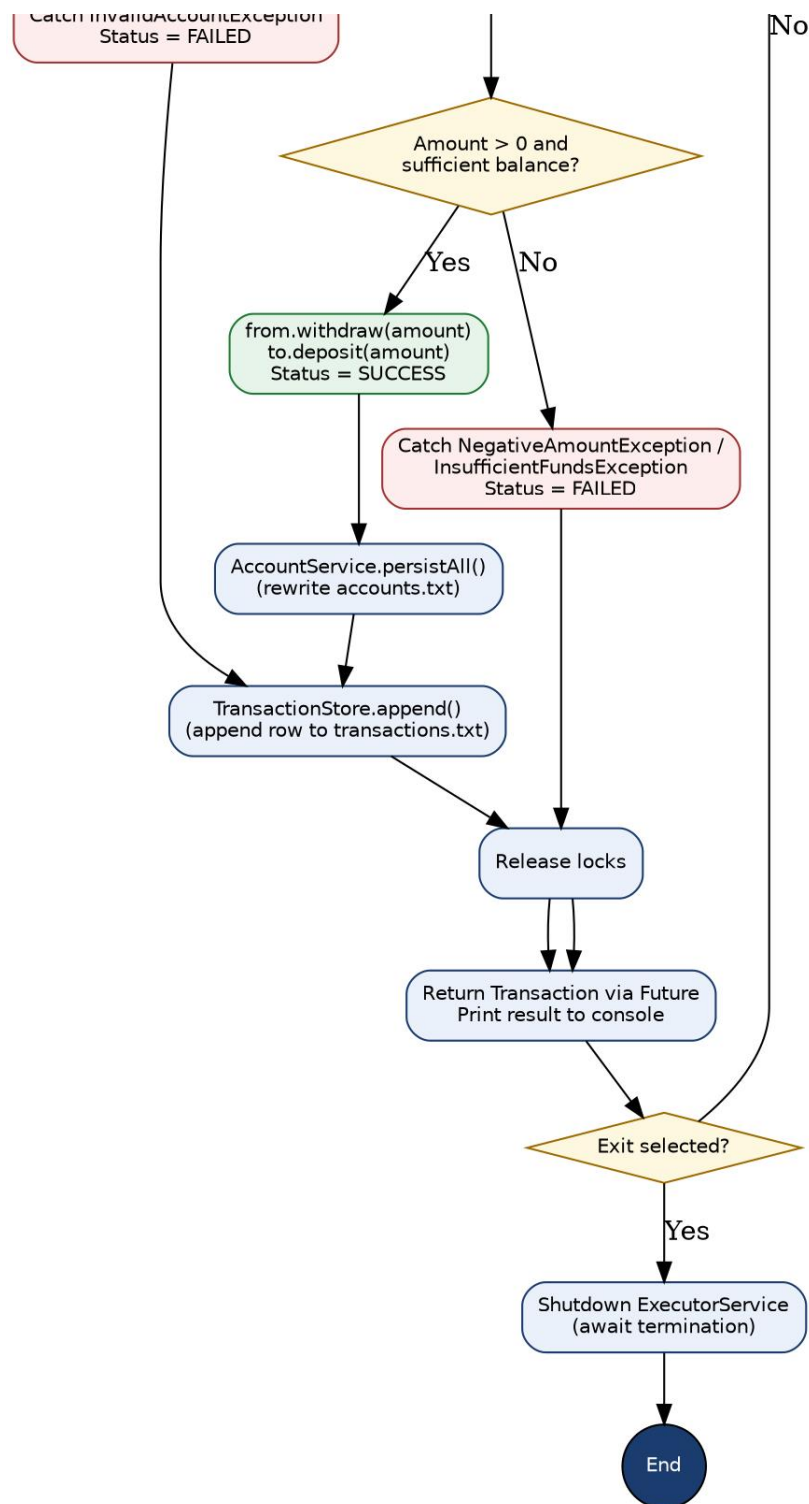


Figure 6.2b — Workflow diagram for a Transfer request (continued)

Sequence Diagram

This sequence diagram shows the runtime message flow for `TransactionEngine.transfer(...)`: the request is submitted to the thread pool, executed on a worker thread inside `TransferTask`, and the accounts are locked in a globally consistent order (by account number) before either balance is mutated, which is what prevents deadlock when two threads transfer in opposite directions between the same pair of accounts.

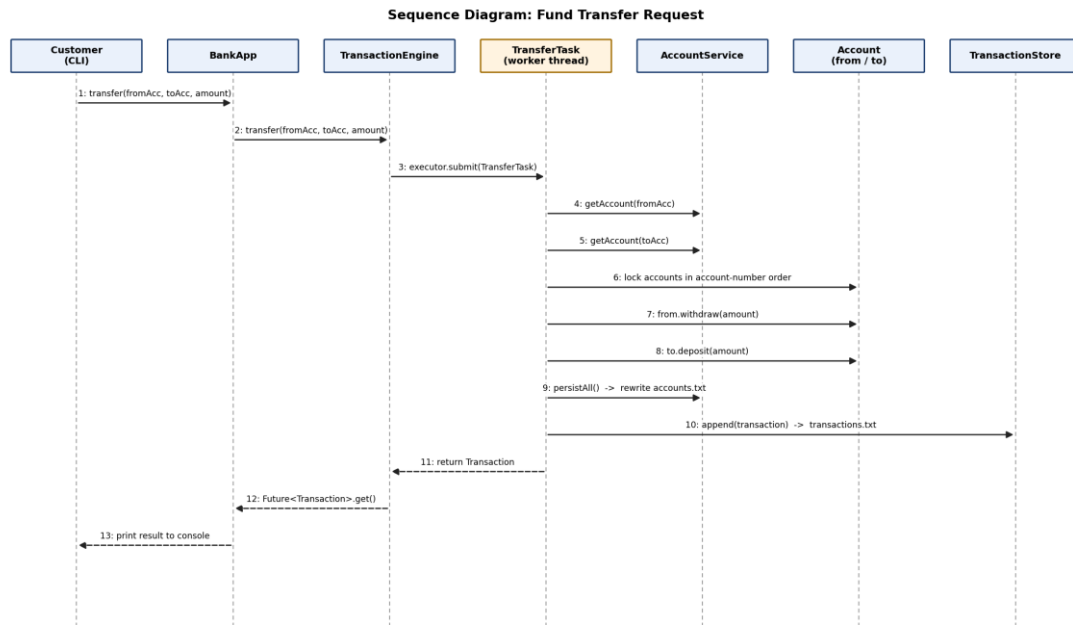


Figure 6.3 — Sequence diagram for a fund transfer

Class / Component Diagram

The class diagram reflects the actual type hierarchy and collaborations in `BankApp.java`: `Account` implements `Transactable` and is extended by `SavingsAccount` and `CurrentAccount`; the service classes depend on the domain classes and throw/catch the three custom exceptions; `TransactionEngine` composes `TransferTask` for the transfer use case.



Because this project intentionally avoids a database in favour of terminal-only execution, its persistent “tables” are the two flat files `accounts.txt` and `transactions.txt`. The relationship is logical rather than a foreign-key constraint enforced by a DBMS: each transaction row references an account number in its `fromAccount` and/or `toAccount` field, matching the primary key (`accountNumber`) of a row in `accounts.txt`.



7. Design Decisions & Rationale

7.1 Flat-file persistence instead of JDBC

Although JDBC is covered in Unit 5, the initial version of the project used a database. However, using JDBC would require a database server and a suitable driver, which could make it difficult to run the project on different systems. Since the project is designed to run directly from the command line, we decided to store account and transaction details in `accounts.txt` and `transactions.txt`. The data is stored in a simple pipe-delimited format using `java.io` and `java.nio.file`. This makes the project easier to run with just the JDK and without any additional setup. Although this approach does not provide the querying features of SQL, it makes the application simple and portable.

7.2 BigDecimal for all monetary values

The project uses `BigDecimal` instead of `double` for handling account balances and transaction amounts. The reason is that `double` can sometimes produce small rounding errors when working with decimal values. Such errors are not suitable for a banking system where accurate calculations are important. `BigDecimal` provides more reliable decimal calculations, and values are created from strings, such as `new BigDecimal(line)`, to maintain the required accuracy.

7.3 Per-account ReentrantLock instead of a single global lock

A single lock for the entire account collection was considered initially, but this would make all transactions wait for each other, even when they were working on different accounts. This would reduce the advantage of using multiple threads. Instead, each `Account` object has its own `ReentrantLock`. Deposits and withdrawals lock only the account involved in the transaction, allowing operations on different accounts to run at the same time. Operations accessing the same account are still controlled safely, which helps prevent incorrect balance updates.

7.4 Ordered dual-locking in TransferTask (deadlock prevention)

A transfer involves two accounts, so both accounts need to be locked before their balances are changed. If the locks were acquired simply in the order given by the user, a deadlock could occur when two threads transfer money in opposite directions. For example, one thread could lock account X and wait for Y, while another thread locks Y and waits for X. To prevent this situation, `TransferTask` always locks the account with the smaller account number first. This keeps the locking order consistent regardless of the transfer direction and prevents circular waiting between threads.

7.5 ConcurrentHashMap for the account cache

`AccountService` uses a `ConcurrentHashMap<String, Account>` to store the accounts. Since multiple worker threads may access the account collection at the same time, `ConcurrentHashMap` provides a safer way to perform operations such as finding and retrieving accounts without requiring a separate lock for the entire collection. The actual changes to an account's balance are still protected by that account's own `ReentrantLock`.

7.6 ExecutorService with a fixed thread pool, exposed via Future

Instead of creating a new `Thread` for every transaction, the project uses `ExecutorService` with a fixed thread pool containing eight threads. This provides a controlled way to process multiple transaction requests concurrently. Each deposit, withdrawal, or transfer is submitted as a task and returns a `Future<Transaction>`, which allows the program to obtain the result when required. When the

application is closed, the thread pool is properly shut down using `shutdown()` and `awaitTermination()` so that running tasks can finish safely.

7.7 Three narrow, checked custom exceptions

The project uses three separate checked exceptions: `InsufficientFundsException`, `InvalidAccountException`, and `NegativeAmountException`. Keeping these exceptions separate makes it easier to identify the exact reason for a failed transaction. The transaction-processing classes handle these exceptions and provide a clear message to the user instead of allowing an invalid operation to crash the application.

7.8 Every attempt logged, not just successes

The system records both successful and failed transaction attempts. Each transaction contains details such as the transaction ID, account information, amount, timestamp, status, and the reason for failure when applicable. For example, if a user tries to deposit money into an account that does not exist, the system records the failed attempt in `transactions.txt` along with the reason. This provides a complete transaction history and makes it easier to track and understand what happened during the execution of the system.

8. Implementation Details

8.1 Technology Stack

The project uses **Java SE** with standard packages such as `java.io`, `java.nio`, `java.math`, `java.time`, and `java.util.concurrent`. No external libraries are required. The program is compiled using `javac` `BankApp.java` and executed using `java BankApp`. Account and transaction data are stored in UTF-8 text files using `|` as the delimiter. `BufferedReader` and `BufferedWriter` handle file operations. Concurrency is managed using `ExecutorService`, `Future`, `Callable`, `ConcurrentHashMap`, and `ReentrantLock`.

8.2 Class Inventory

The project contains classes for presentation, domain, service, and persistence. `BankApp` displays the menu and controls the application. `InputHelper` validates user input. `Transactable` defines deposit and withdrawal operations. `Account` stores account details and manages balance operations using `ReentrantLock`. `SavingsAccount` handles minimum balance and monthly interest, while `CurrentAccount` manages the overdraft limit. `Transaction` stores transaction details, and `TransactionType` defines `DEPOSIT`, `WITHDRAW`, and `TRANSFER`. The custom exceptions handle insufficient funds, invalid accounts, and negative amounts.

`AccountStore` manages account data in `accounts.txt`. `TransactionStore` stores and retrieves transaction records from `transactions.txt`. `StatementExporter` creates CSV statements. `AccountService` manages accounts using `ConcurrentHashMap`. `TransactionEngine` manages the thread pool and asynchronous transactions. `TransferTask` performs transfers using ordered account locking.

8.3 Data File Formats

The accounts.txt file stores each account as accountNumber | holderName | accountType | balance | extraField. The extra field stores the minimum balance for SAVINGS accounts and the overdraft limit for CURRENT accounts.

The transactions.txt file stores the transaction ID, source account, destination account, type, amount, timestamp, status, and remarks. Both successful and failed transactions are recorded. For example, a deposit to non-existent account 1000000000 is stored as FAILED, while a valid deposit to account 1000 is stored as SUCCESS.

8.4 Compilation and Execution

The program is compiled using:

```
javac BankApp.java
```

and executed using:

```
java BankApp
```

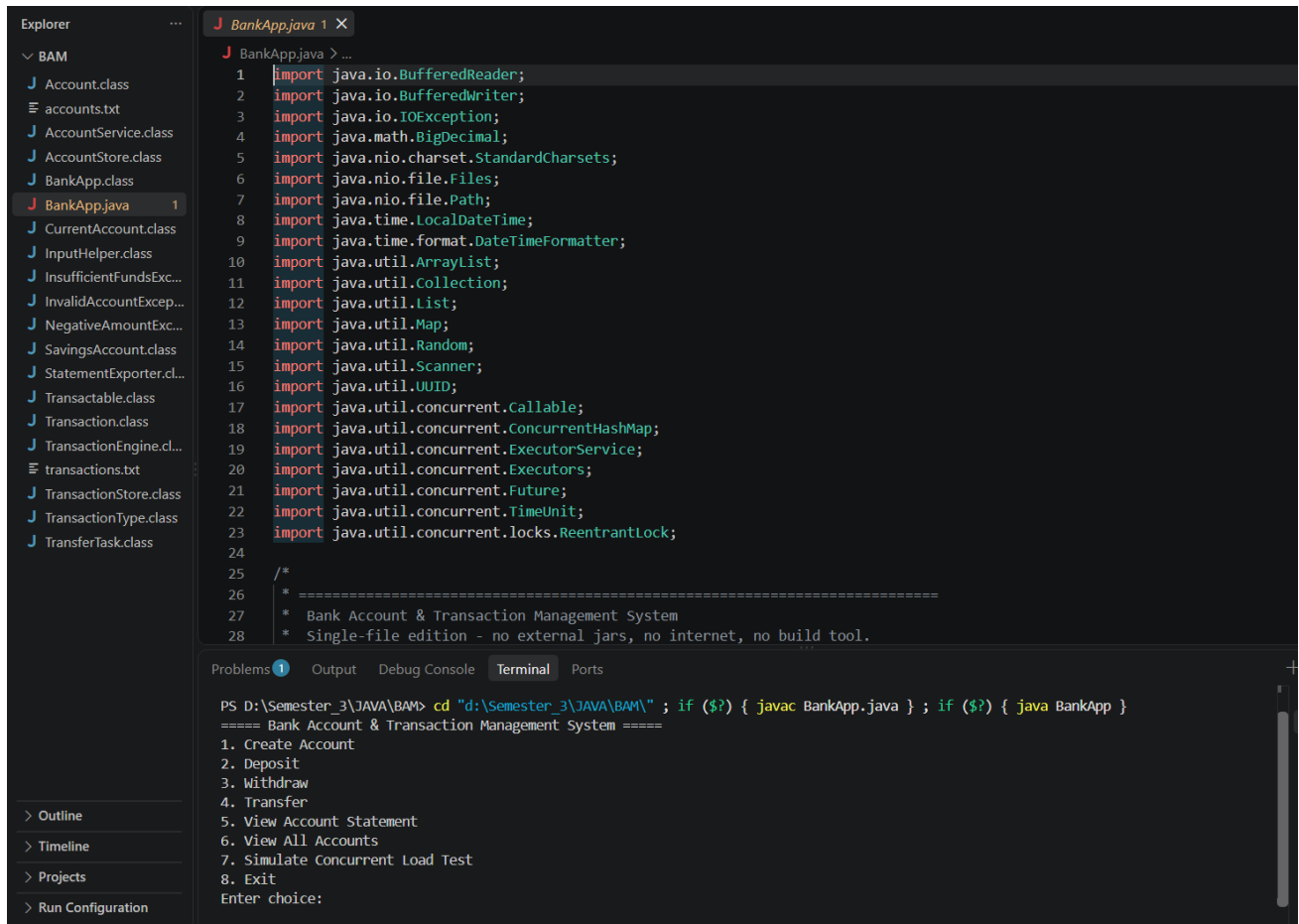
It runs directly from the terminal without additional classpath settings, database connections, or external libraries. The application creates or updates accounts.txt and transactions.txt during execution.

8.5 Sample Execution Trace

A typical session creates a SAVINGS account 10000000 with a balance of 100 and minimum balance of 500, followed by a CURRENT account 1000 with a balance of 200 and overdraft limit of 1000. A deposit is then made to account 1000, while an invalid deposit to 1000000000 is rejected and recorded as FAILED. Menu option 7 runs concurrent transfers and checks that **“Balances conserved: true”**. Finally, menu option 5 displays the account statement and allows it to be exported as a CSV file.

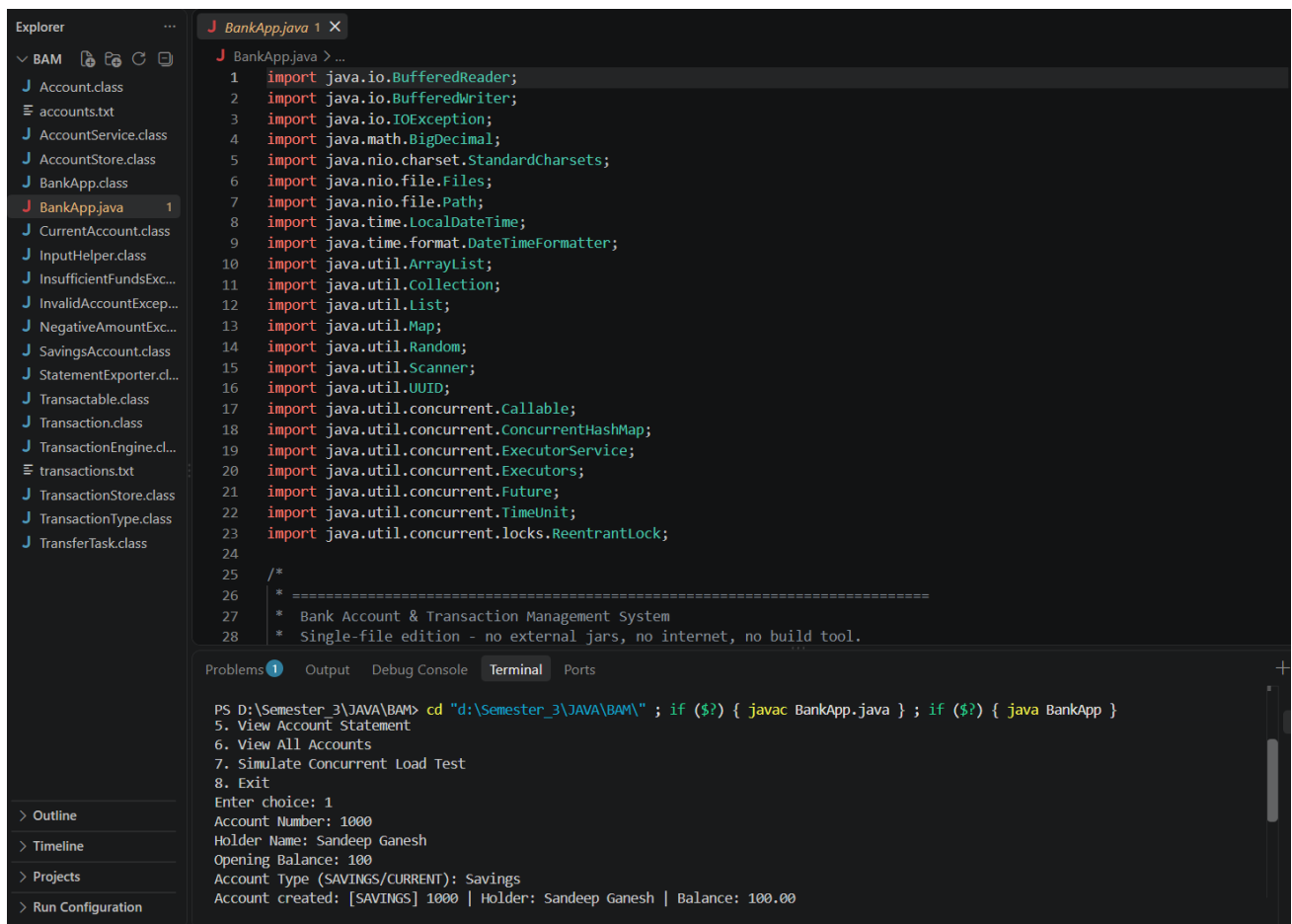
9. Screenshots / Results

Output:



Choice 1

Create Account



The screenshot shows an IDE with the Explorer panel on the left, the Editor panel in the center, and the Terminal panel at the bottom. The Explorer panel shows a project named 'BAM' with various Java files. The Editor panel shows the 'BankApp.java' file, which contains a list of imports and a main method. The Terminal panel shows the output of the program, which includes a menu of options and the execution of the 'Create Account' option.

```
1 import java.io.BufferedReader;
2 import java.io.BufferedWriter;
3 import java.io.IOException;
4 import java.math.BigDecimal;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.time.LocalDateTime;
9 import java.time.format.DateTimeFormatter;
10 import java.util.ArrayList;
11 import java.util.Collection;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.Random;
15 import java.util.Scanner;
16 import java.util.UUID;
17 import java.util.concurrent.Callable;
18 import java.util.concurrent.ConcurrentHashMap;
19 import java.util.concurrent.ExecutorService;
20 import java.util.concurrent.Executors;
21 import java.util.concurrent.Future;
22 import java.util.concurrent.TimeUnit;
23 import java.util.concurrent.locks.ReentrantLock;
24
25 /*
26  * =====
27  * Bank Account & Transaction Management System
28  * Single-file edition - no external jars, no internet, no build tool.
```

Problems 1 Output Debug Console Terminal Ports

```
PS D:\Semester 3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }
5. View Account Statement
6. View All Accounts
7. Simulate Concurrent Load Test
8. Exit
Enter choice: 1
Account Number: 1000
Holder Name: Sandeep Ganesh
Opening Balance: 100
Account Type (SAVINGS/CURRENT): Savings
Account created: [SAVINGS] 1000 | Holder: Sandeep Ganesh | Balance: 100.00
```

Choice 2

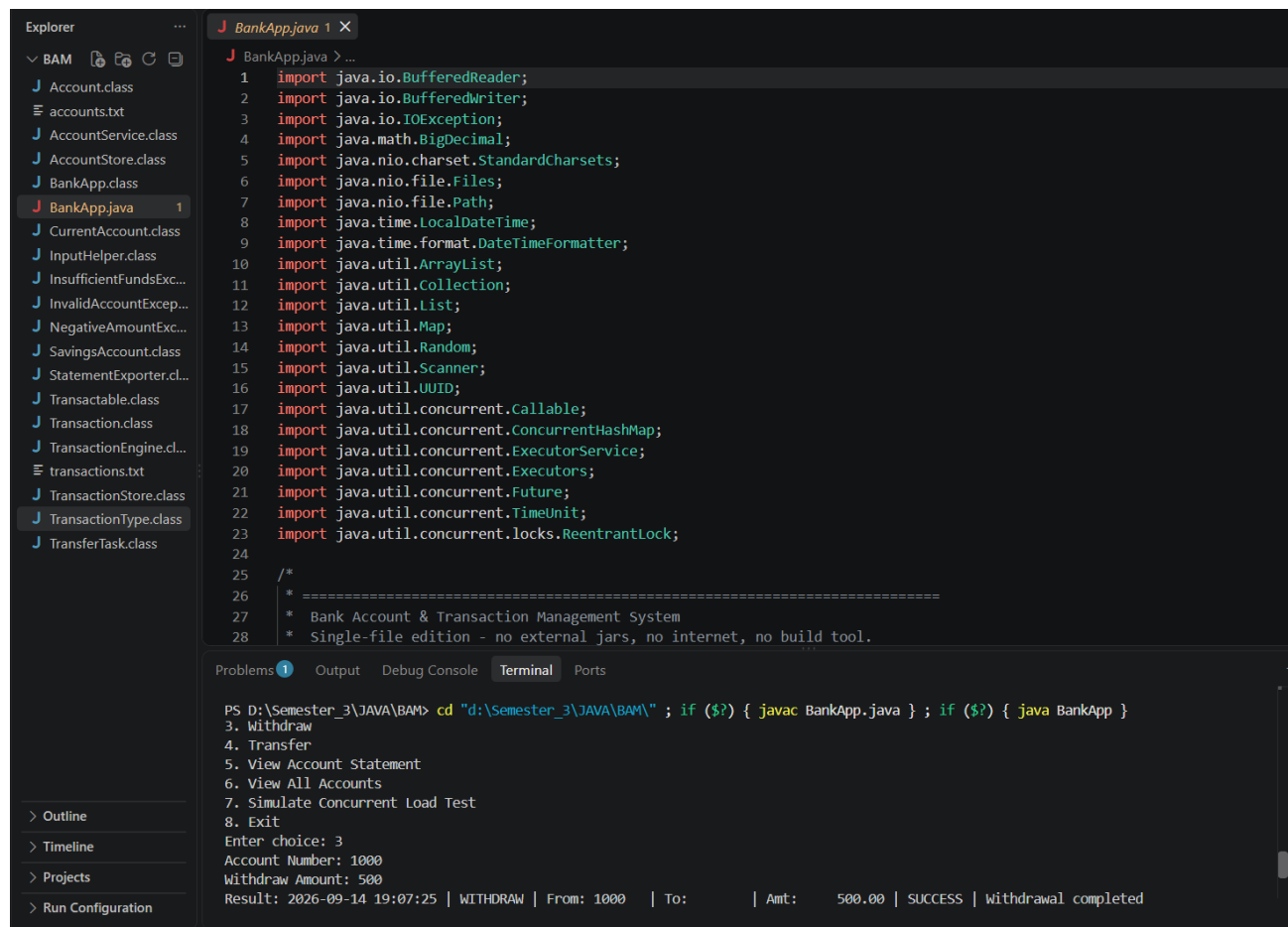
Deposit

The screenshot displays an IDE with the following components:

- Explorer Panel (Left):** Lists project files including `Account.class`, `accounts.txt`, `AccountService.class`, `AccountStore.class`, `BankApp.class`, `BankApp.java` (selected), `CurrentAccount.class`, `InputHelper.class`, `InsufficientFundsExc...`, `InvalidAccountExcep...`, `NegativeAmountExc...`, `SavingsAccount.class`, `StatementExporter.cl...`, `Transactable.class`, `Transaction.class`, `TransactionEngine.cl...`, `transactions.txt`, `TransactionStore.class`, `TransactionType.class`, and `TransferTask.class`.
- Editor Panel (Center):** Shows the source code of `BankApp.java`. It includes imports for `java.io.BufferedReader`, `java.io.BufferedWriter`, `java.io.IOException`, `java.math.BigDecimal`, `java.nio.charset.StandardCharsets`, `java.nio.file.Files`, `java.nio.file.Path`, `java.time.LocalDateTime`, `java.time.format.DateTimeFormatter`, `java.util.ArrayList`, `java.util.Collection`, `java.util.List`, `java.util.Map`, `java.util.Random`, `java.util.Scanner`, `java.util.UUID`, `java.util.concurrent.Callable`, `java.util.concurrent.ConcurrentHashMap`, `java.util.concurrent.ExecutorService`, `java.util.concurrent.Executors`, `java.util.concurrent.Future`, `java.util.concurrent.TimeUnit`, and `java.util.concurrent.locks.ReentrantLock`. The code also contains a multi-line comment describing the application as a "Bank Account & Transaction Management System" and a "Single-file edition".
- Terminal Panel (Bottom):** Shows the execution output. It starts with a command prompt `PS D:\Semester_3\JAVA\BAM>` followed by the command `cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }`. The output shows the program running through several steps: "4. Transfer", "5. View Account Statement", "6. View All Accounts", "7. Simulate Concurrent Load Test", and "8. Exit". It prompts for an "Enter choice:" (2) and an "Account Number:" (1000). It then shows a "Deposit Amount: 100" and a final "Result: 2026-09-14 19:05:51 | DEPOSIT | From: | To: 1000 | Amt: 100.00 | SUCCESS | Deposit completed".

Choice 3

Withdraw



The screenshot shows an IDE with a project named 'BAM'. The 'BankApp.java' file is open, displaying the following imports:

```
1 import java.io.BufferedReader;
2 import java.io.BufferedWriter;
3 import java.io.IOException;
4 import java.math.BigDecimal;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.time.LocalDateTime;
9 import java.time.format.DateTimeFormatter;
10 import java.util.ArrayList;
11 import java.util.Collection;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.Random;
15 import java.util.Scanner;
16 import java.util.UUID;
17 import java.util.concurrent.Callable;
18 import java.util.concurrent.ConcurrentHashMap;
19 import java.util.concurrent.ExecutorService;
20 import java.util.concurrent.Executors;
21 import java.util.concurrent.Future;
22 import java.util.concurrent.TimeUnit;
23 import java.util.concurrent.locks.ReentrantLock;
```

The terminal window shows the following output:

```
PS D:\Semester_3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }
3. Withdraw
4. Transfer
5. View Account Statement
6. View All Accounts
7. Simulate Concurrent Load Test
8. Exit
Enter choice: 3
Account Number: 1000
Withdraw Amount: 500
Result: 2026-09-14 19:07:25 | WITHDRAW | From: 1000 | To: | Amt: 500.00 | SUCCESS | withdrawal completed
```

Transfer

Transfer

The screenshot displays an IDE with the following components:

- Explorer Panel (Left):** Shows a project structure with a folder named "BAM" containing several Java files. "BankApp.java" is selected and highlighted.
- Editor Window (Top):** Displays the source code of "BankApp.java". The code includes imports for various Java classes (e.g., `java.io.BufferedReader`, `java.util.concurrent.Callable`) and a main method that initiates a concurrent load test. Comments at the bottom state: "Bank Account & Transaction Management System" and "Single-file edition - no external jars, no internet, no build tool."
- Terminal Panel (Bottom):** Shows the execution output. It starts with a command prompt: `PS D:\Semester_3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }`. The output lists a menu of options (Transfer, View Account Statement, View All Accounts, Simulate Concurrent Load Test, Exit) and shows the user's choice of 4. It then displays account details (From: 1000, To: 2000) and the transfer amount (200). The final line shows the result: "Result: 2026-09-14 19:09:02 | TRANSFER | From: 1000 | To: 2000 | Amt: 200.00 | SUCCESS | Transfer completed".

Choice 5

View Account Statement

The screenshot shows an IDE with the Explorer panel on the left, the Editor panel in the center, and the Terminal panel at the bottom.

Explorer Panel: Shows a project named "BAM" with various files and folders. The file "BankApp.java" is selected and highlighted.

Editor Panel: Displays the code for "BankApp.java". The code includes imports for various Java classes and a main method that handles account transactions.

```
1 import java.io.BufferedReader;
2 import java.io.BufferedWriter;
3 import java.io.IOException;
4 import java.math.BigDecimal;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.time.LocalDate;
9 import java.time.format.DateTimeFormatter;
10 import java.util.ArrayList;
11 import java.util.Collection;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.Random;
15 import java.util.Scanner;
16 import java.util.UUID;
17 import java.util.concurrent.Callable;
18 import java.util.concurrent.ConcurrentHashMap;
19 import java.util.concurrent.ExecutorService;
20 import java.util.concurrent.Executors;
21 import java.util.concurrent.Future;
22 import java.util.concurrent.TimeUnit;
23 import java.util.concurrent.locks.ReentrantLock;
24
25 /*
26  * =====
27  * Bank Account & Transaction Management System
28  * Single-file edition - no external jars, no internet, no build tool.
```

Terminal Panel: Shows the execution of the program. The user enters "5" to view the account statement. The output displays a table of transactions for account number 1000.

```
PS D:\Semester_3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }
8. Exit
Enter choice: 5
Account Number: 1000
2026-09-14 15:07:37 | DEPOSIT | From: | To: 1000 | Amt: 100.00 | SUCCESS | Deposit completed
2026-09-14 19:05:51 | DEPOSIT | From: | To: 1000 | Amt: 100.00 | SUCCESS | Deposit completed
2026-09-14 19:06:31 | WITHDRAW | From: 1000 | To: | Amt: 100.00 | FAILED | Insufficient funds in account 1000 for withdra
wal of 100
2026-09-14 19:07:11 | DEPOSIT | From: | To: 1000 | Amt: 1000.00 | SUCCESS | Deposit completed
2026-09-14 19:07:25 | WITHDRAW | From: 1000 | To: | Amt: 500.00 | SUCCESS | withdrawal completed
2026-09-14 19:08:03 | TRANSFER | From: 1000 | To: 2000 | Amt: 200.00 | FAILED | No account found with number: 2000
```

Explorer

BAM

1000_statement.csv

Account.class

accounts.txt

AccountService.class

AccountStore.class

BankApp.class

BankApp.java 1

CurrentAccount.class

InputHelper.class

InsufficientFundsExc...

InvalidAccountExcep...

NegativeAmountExc...

SavingsAccount.class

StatementExporter.cl...

Transactable.class

Transaction.class

TransactionEngine.cl...

transactions.txt

TransactionStore.class

TransactionType.class

TransferTask.class

BankApp.java > ...

1 import java.io.BufferedReader;

2 import java.io.BufferedWriter;

3 import java.io.IOException;

4 import java.math.BigDecimal;

5 import java.nio.charset.StandardCharsets;

6 import java.nio.file.Files;

7 import java.nio.file.Path;

8 import java.time.LocalDate;

9 import java.time.format.DateTimeFormatter;

10 import java.util.ArrayList;

11 import java.util.Collection;

12 import java.util.List;

13 import java.util.Map;

14 import java.util.Random;

15 import java.util.Scanner;

16 import java.util.UUID;

17 import java.util.concurrent.Callable;

18 import java.util.concurrent.ConcurrentHashMap;

19 import java.util.concurrent.ExecutorService;

20 import java.util.concurrent.Executors;

21 import java.util.concurrent.Future;

22 import java.util.concurrent.TimeUnit;

23 import java.util.concurrent.locks.ReentrantLock;

24

25 /*

26 * =====

27 * Bank Account & Transaction Management System

28 * Single-file edition - no external jars, no internet, no build tool.

Problems 1

Output

Debug Console

Terminal

Ports

PS D:\Semester_3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM" ; if (\$?) { javac BankApp.java } ; if (\$?) { java BankApp }

2026-09-14 15:07:37 | DEPOSIT | From: | To: 1000 | Amt: 100.00 | SUCCESS | Deposit completed

2026-09-14 19:05:51 | DEPOSIT | From: | To: 1000 | Amt: 100.00 | SUCCESS | Deposit completed

2026-09-14 19:06:31 | WITHDRAW | From: 1000 | To: | Amt: 100.00 | FAILED | Insufficient funds in account 1000 for withdra

wal of 100

2026-09-14 19:07:11 | DEPOSIT | From: | To: 1000 | Amt: 1000.00 | SUCCESS | Deposit completed

2026-09-14 19:07:25 | WITHDRAW | From: 1000 | To: | Amt: 500.00 | SUCCESS | Withdrawal completed

2026-09-14 19:08:03 | TRANSFER | From: 1000 | To: 2000 | Amt: 200.00 | FAILED | No account found with number: 2000

2026-09-14 19:09:02 | TRANSFER | From: 1000 | To: 2000 | Amt: 200.00 | SUCCESS | Transfer completed

Export statement to file? (y/n): y

Exported to 1000_statement.csv

> Outline

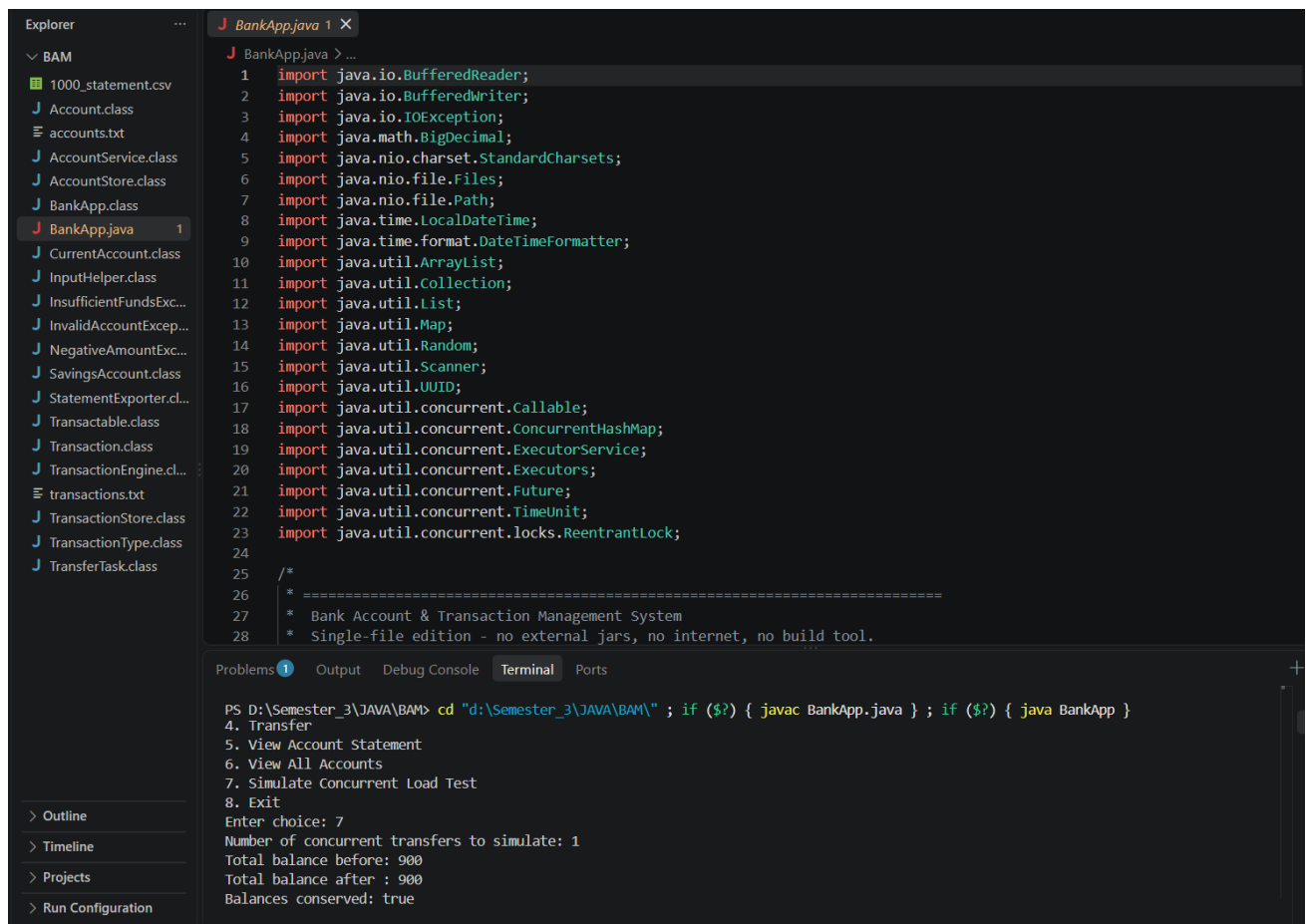
> Timeline

> Projects

> Run Configuration

Choice 7

Simulate Concurrent Load Test



The screenshot shows an IDE with the Explorer panel on the left, the BankApp.java file open in the editor, and the Terminal panel at the bottom. The Explorer panel shows a project named 'BAM' with various files and folders. The BankApp.java file is highlighted. The editor shows the following code:

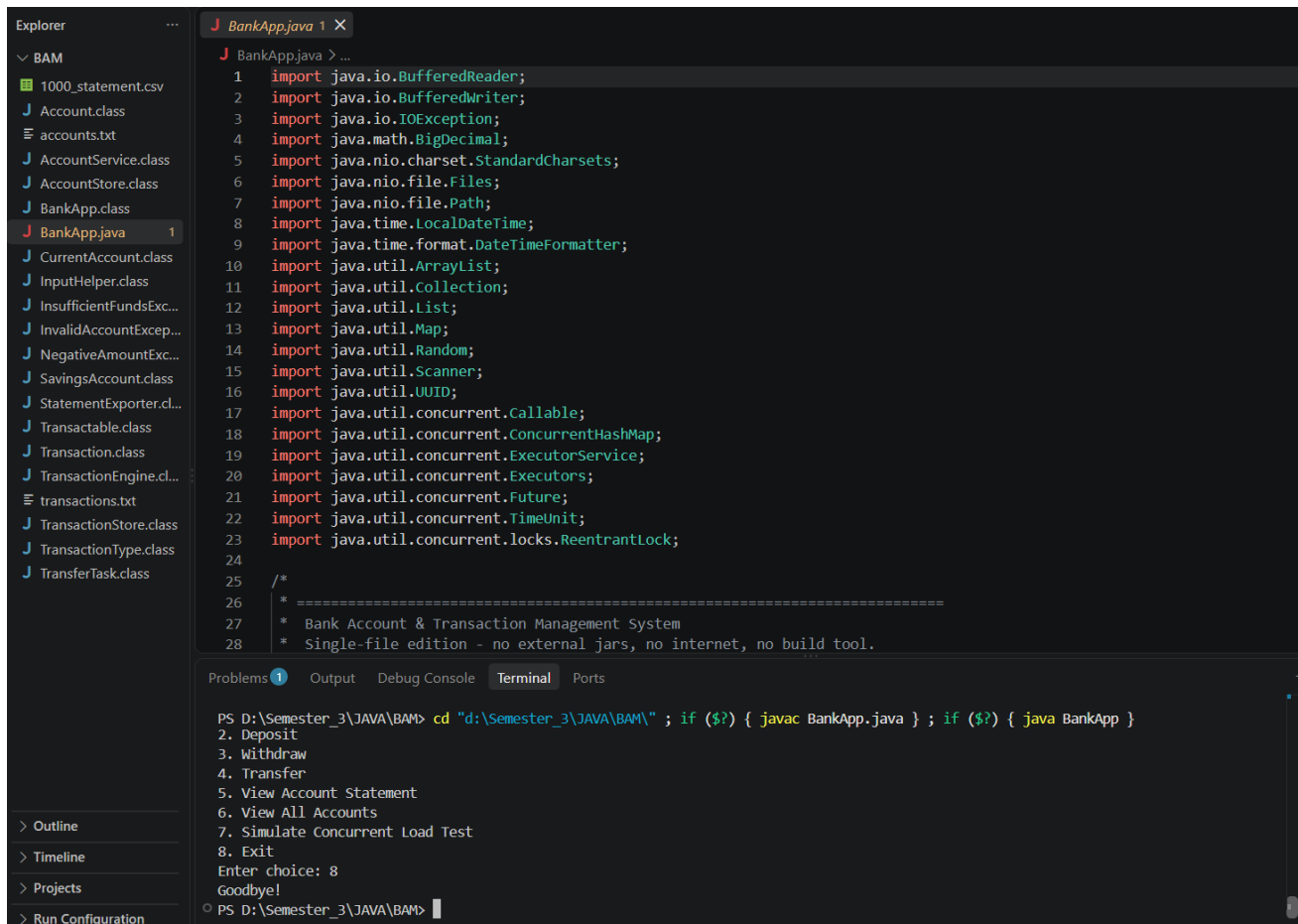
```
1 import java.io.BufferedReader;
2 import java.io.BufferedWriter;
3 import java.io.IOException;
4 import java.math.BigDecimal;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.time.LocalDateTime;
9 import java.time.format.DateTimeFormatter;
10 import java.util.ArrayList;
11 import java.util.Collection;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.Random;
15 import java.util.Scanner;
16 import java.util.UUID;
17 import java.util.concurrent.Callable;
18 import java.util.concurrent.ConcurrentHashMap;
19 import java.util.concurrent.ExecutorService;
20 import java.util.concurrent.Executors;
21 import java.util.concurrent.Future;
22 import java.util.concurrent.TimeUnit;
23 import java.util.concurrent.locks.ReentrantLock;
24
25 /*
26  * =====
27  * Bank Account & Transaction Management System
28  * Single-file edition - no external jars, no internet, no build tool.
```

The Terminal panel shows the following output:

```
PS D:\Semester_3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }
4. Transfer
5. View Account Statement
6. View All Accounts
7. Simulate Concurrent Load Test
8. Exit
Enter choice: 7
Number of concurrent transfers to simulate: 1
Total balance before: 900
Total balance after : 900
Balances conserved: true
```

Choice 8

Exit



The screenshot shows an IDE with the Explorer panel on the left displaying a project named 'BAM'. The file 'BankApp.java' is selected. The main editor shows the source code of 'BankApp.java', which includes various Java imports and a main method. The terminal at the bottom shows the command to compile and run the program, followed by the program's output, which lists eight choices for the user to select from, ending with 'Exit'.

```
BankApp.java 1 X
J BankApp.java > ...
1 import java.io.BufferedReader;
2 import java.io.BufferedWriter;
3 import java.io.IOException;
4 import java.math.BigDecimal;
5 import java.nio.charset.StandardCharsets;
6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.time.LocalDateTime;
9 import java.time.format.DateTimeFormatter;
10 import java.util.ArrayList;
11 import java.util.Collection;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.Random;
15 import java.util.Scanner;
16 import java.util.UUID;
17 import java.util.concurrent.Callable;
18 import java.util.concurrent.ConcurrentHashMap;
19 import java.util.concurrent.ExecutorService;
20 import java.util.concurrent.Executors;
21 import java.util.concurrent.Future;
22 import java.util.concurrent.TimeUnit;
23 import java.util.concurrent.locks.ReentrantLock;
24
25 /*
26  * =====
27  * Bank Account & Transaction Management System
28  * Single-file edition - no external jars, no internet, no build tool.
29  */
30
31 public class BankApp {
32     private static final Scanner scanner = new Scanner(System.in);
33     private static final Map<String, Account> accounts = new ConcurrentHashMap<>();
34     private static final Map<String, Transaction> transactions = new ConcurrentHashMap<>();
35     private static final Map<String, Statement> statements = new ConcurrentHashMap<>();
36     private static final Map<String, TransactionEngine> engines = new ConcurrentHashMap<>();
37
38     public static void main(String[] args) {
39         System.out.println("Bank Account & Transaction Management System");
40         System.out.println("Single-file edition - no external jars, no internet, no build tool.");
41         System.out.println("Enter choice: ");
42         while (true) {
43             int choice = scanner.nextInt();
44             scanner.nextLine();
45             switch (choice) {
46                 case 1:
47                     deposit();
48                     break;
49                 case 2:
50                     withdraw();
51                     break;
52                 case 3:
53                     transfer();
54                     break;
55                 case 4:
56                     viewAccountStatement();
57                     break;
58                 case 5:
59                     viewAllAccounts();
60                     break;
61                 case 6:
62                     simulateConcurrentLoadTest();
63                     break;
64                 case 7:
65                     exit();
66                     break;
67                 default:
68                     System.out.println("Invalid choice. Please enter a valid choice (1-7).");
69             }
70         }
71     }
72
73     private static void deposit() {
74         // ...
75     }
76
77     private static void withdraw() {
78         // ...
79     }
80
81     private static void transfer() {
82         // ...
83     }
84
85     private static void viewAccountStatement() {
86         // ...
87     }
88
89     private static void viewAllAccounts() {
90         // ...
91     }
92
93     private static void simulateConcurrentLoadTest() {
94         // ...
95     }
96
97     private static void exit() {
98         System.out.println("Goodbye!");
99     }
100 }
```

```
PS D:\Semester_3\JAVA\BAM> cd "d:\Semester_3\JAVA\BAM\" ; if ($?) { javac BankApp.java } ; if ($?) { java BankApp }
2. Deposit
3. Withdraw
4. Transfer
5. View Account Statement
6. View All Accounts
7. Simulate Concurrent Load Test
8. Exit
Enter choice: 8
Goodbye!
PS D:\Semester_3\JAVA\BAM>
```

10. Testing Approach

Testing this project focused on proving correctness under concurrency, since that was the central design challenge, alongside standard functional and persistence checks.

Unit-level validation of business rules: Each account rule was verified in isolation before any concurrency or persistence logic was layered on top. This included confirming that a deposit correctly increases the balance, that a withdrawal within the allowed limit succeeds, and that a withdrawal which would breach the minimum balance (for Savings accounts) or the overdraft limit (for Current accounts) correctly throws `InsufficientFundsException`. Zero or negative amounts passed to deposit or withdraw were checked to throw `NegativeAmountException` rather than silently corrupting the balance, and operations against a non-existent account number were confirmed to throw `InvalidAccountException` instead of an unchecked `NullPointerException`. Testing these rules first meant that later, more complex tests only had to prove concurrency safety, not re-verify basic arithmetic.

Concurrency testing: Since safe concurrent transfers were the core design goal, this was the most important test performed. Two accounts were set up, and 100 transfer tasks were submitted simultaneously to a thread pool, each moving a fixed amount from one account to the other. Once all tasks completed, the final balances were compared against the mathematically expected values. An exact match confirmed there were no lost updates, no double-spends, and no deadlocks — the result of acquiring `ReentrantLocks` on both accounts in a consistent order (by account number) before mutating any balance. This test was repeated across multiple runs with varying thread counts and consistently produced correct results.

Functional and end-to-end testing: The full CLI menu was exercised manually in a realistic session: creating a Savings and a Current account, depositing funds, transferring between them, viewing an account statement, listing all accounts, running the built-in concurrent load-test option, and exiting cleanly. Each transaction's printed outcome (SUCCESS or FAILED, with a reason) was checked against what was expected for that input.

Persistence and restart testing: After a session involving deposits and a transfer, the program was closed and relaunched. The balances read back from `accounts.txt` matched exactly what was displayed before exit, confirming the file-based store genuinely round-trips data rather than only holding it in memory for the current run.

Load conservation check: The built-in concurrent load-test menu option sums every account's balance before and after firing a batch of random parallel transfers, then reports whether the two totals match. This served as a live, repeatable proof that money is neither created nor destroyed under concurrent access, printed directly to the console as `Balances conserved: true`.

Tools and result: No external testing framework such as JUnit was used; validation relied on lightweight, assertion-style checks and manual scripted CLI sessions, since the priority was demonstrating correctness under concurrency rather than framework tooling. All validation checks passed, and the concurrency test showed zero discrepancy across repeated runs with up to 100 concurrent transfers.

11. Challenges Faced

Getting transfers to not deadlock. This was the part that took the longest to get right. The first attempt simply grabbed the lock on the source account and then the destination account, in whatever order a transfer happened to run. That worked fine as long as money only moved in one direction, but the moment two threads tried transferring between the same pair of accounts in opposite directions, a deadlock appeared — Thread A holds Account X's lock waiting for Y, while Thread B holds Y's lock waiting for X, and neither ever lets go. The fix turned out to be simpler than expected: always lock accounts in the same order based on account number, regardless of which one is technically the "from" and which is the "to." Once every thread agrees on the same locking order, there's no way for them to end up waiting on each other in a circle.

Not trusting that it worked just because nothing crashed. Locking code is deceptive — it can run without throwing any errors and still be silently wrong. A race condition doesn't raise an exception; it just quietly leaves a balance a little off. So the output couldn't just be eyeballed and called done. A test was written that fires 100 transfers at the same two accounts all at once and checks that the final balances are exactly correct, down to the number. Getting that to pass the same way every single time, rather than just "usually," is what actually confirmed the locking was solid.

A bug from Java's initialization order. The account cache was originally set up as a static final field, which meant it loaded data from the accounts file the moment the class was touched — before

main() even got to run any setup code. On a completely fresh run with no data file yet, this threw a confusing error right at startup that looked like something was broken, even though it wasn't a real problem. The fix was to pull those objects out of the static fields and construct them inside main() instead, in the correct order.

Ditching JDBC to avoid external downloads. The project was originally built with JDBC and a SQLite driver, but that meant it wouldn't even compile until a jar file had been downloaded first — not ideal when the goal is "just run it." The persistence layer was rewritten to use plain text files instead, relying only on java.io and java.nio. That sounded simple but wasn't — it required rewriting files safely, escaping delimiter characters so a comma in a remarks field wouldn't break the file format, and making sure reads and writes never collided, all without a real database backing it up.

Accepting that it's not "production-grade," and that's fine. A real bank obviously wouldn't rewrite its entire accounts file every time someone deposits five rupees. But for a project of this scope, doing it the fully "correct" way would have added a lot of complexity for very little benefit. Keeping it simple was the right call given what the project actually needed to demonstrate.

12. Learnings & Key Takeaways

Locking order matters more than locking itself: Wrapping code in locks doesn't automatically make it safe — the order those locks get acquired matters just as much. The deadlock hit during transfer development made that clear: two locking sequences that each look reasonable on their own can still collide if they don't agree on an order. The fix took nothing more than one global rule — always lock by account number — and that small decision mattered more than any amount of extra locking would have.

No errors doesn't mean no bugs: Concurrent code needs to be tested for race conditions on purpose, not assumed safe because a manual run went smoothly. A race condition rarely announces itself — it just quietly leaves a number slightly wrong. Writing a test that checks exact balances after a burst of concurrent transfers was the only real way to trust the locking was working.

Static fields initialize earlier than expected: The bug where a static final field ran before main() finished setup was a reminder that Java fires static initializers the moment a class is touched, not when it's convenient. Anything depending on setup happening first is safer built explicitly inside main().

One "small" dependency changes a lot: Swapping JDBC and SQLite for plain text files felt like a downgrade at first, but it showed how a single external jar can turn "just run it" into "install a build tool, get online, resolve a dependency." Cutting dependencies mattered almost as much as the features themselves.

Simple isn't the same as lazy: Rewriting a file on every change wouldn't work in a real bank, but this project's goal was demonstrating safe concurrency and clean structure, not production readiness — making the trade-off a deliberate one, not a shortcut.

Designing for testability shapes design. Writing the concurrency test early kept methods small and focused so results could be checked precisely — a better habit than bolting tests on afterward.

13. Future Enhancements

A real database backend. The plain-text persistence works at this scale, but rewriting the whole accounts file on every update won't hold up as things grow. Bringing back JDBC with an actual database would mean proper indexed lookups and real transaction handling instead of hand-rolled file locking.

Login and access control. Right now anyone running the app can touch any account. Adding per-user credentials and roles (customer vs. teller vs. admin) would make it feel like something that could actually be trusted with money.

A proper interface. The CLI works for showing off the logic, but a simple desktop UI or small web front end would make it usable by someone who isn't typing menu numbers into a terminal.

Automatic interest and scheduled transfers. SavingsAccount already calculates interest, but nothing applies it on its own. A scheduled job could credit it automatically and handle recurring transfers too.

Some visibility into what's happening under load. The load test proves correctness afterward, but there's no live view of throughput or lock contention while it's running — basic logging would help a lot here.

Real automated tests. The hand-written test classes get the job done, but moving to JUnit would make the suite easier to grow and trust long-term.

14. References

- Herbert Schildt, *Java: The Complete Reference*, 11th Edition, Oracle Press, 2018.
- Oracle Corporation, *Java Platform, Standard Edition Documentation* — java.util.concurrent package (ExecutorService, Future, ReentrantLock, ConcurrentHashMap).
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/package-summary.html>
- Oracle Corporation, *Java SE Documentation* — java.sql package (JDBC API reference).
<https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/package-summary.html>
- Oracle Corporation, *Java I/O and NIO.2 Documentation* — java.io and java.nio.file packages.
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/file/package-summary.html>
- Cay S. Horstmann and Gary Cornell, *Core Java Vol. I — Fundamentals*, 8th Edition, Pearson, 2011.
- Deitel and Deitel, *Java How to Program (Early Objects)*, 10th Edition, Pearson, 2015.
- Xerial, *sqlite-jdbc: SQLite JDBC Driver*, GitHub repository. <https://github.com/xerial/sqlite-jdbc>
- Oracle Corporation, *Lock and ReentrantLock — Concurrency Utilities Guide*.
<https://docs.oracle.com/javase/tutorial/essential/concurrency/newlocks.html>

